13 DE NOVIEMBRE DE 2026

Componentes del catálogo

Tarjetas, rejilla, filtros y los cuatro estados, con datos de prueba que tienen la forma de la API



Programación IV — Frontend · UNIAJC · William Henao

HOY

Qué páginas desarrollamos hoy

`/catalogo`

Catálogo

Filtros, buscador, conteo, seis tarjetas y paginación. Igual al mockup.

`/`

Inicio

Las tres recetas destacadas reemplazan el recuadro punteado.

`/estados`

Estados (nueva)

Referencia de los cuatro estados: cargando, con datos, vacío y error.

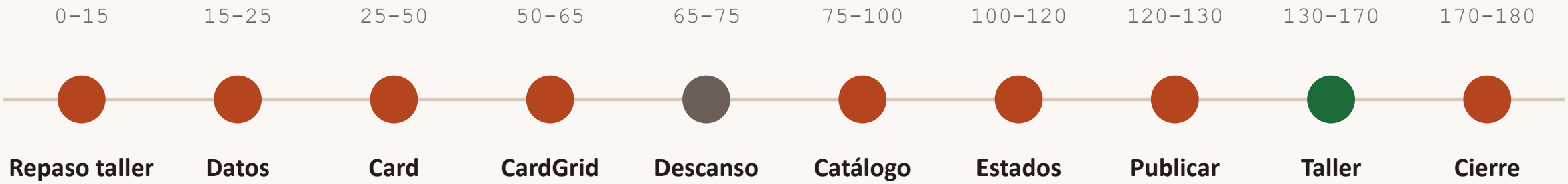
Componentes nuevos

`datos-de-prueba.js` `Card` `CardGrid` `BarraDeBusqueda` `Filtros` `Paginacion` `Estado` `Esqueleto`

Todavía sin JavaScript: los filtros, el buscador y la paginación se ven, pero no funcionan.

AGENDA

Plan de las tres horas



Dos errores intencionales

Una imagen sin width y height (el contenido salta) y una columna sin min-width: 0 (la página se desplaza de lado a 320 px).

Primero datos falsos, con la forma de los reales

```
export const recetas = [
  {
    id: "52772",
    nombre: "Pollo teriyaki al horno",
    categoria: "Pollo",
    area: "Japonesa",
    imagen: "img/recetas/pollo.svg",
    minutos: 45,
    dificultad: "Media",
  },
  // ... cinco más
];
```

```
new Set(recetas.map(r => r.categoria))
```

Misma forma

Los mismos campos que vamos a sacar de la API. Cuando lleguen los datos reales, las tarjetas no cambian.

Seis elementos

Suficientes para ver la rejilla en dos filas y probar la paginación.

new Set()

Quita repetidos: "Vegetariana" aparece dos veces y en los filtros queda una.

Anatomía de una tarjeta

```
---
const { receta } =
  Astro.props;
const { nombre,
  minutos, ... } = receta;
---
```



Tiempo sobre la imagen

Es el dato que decide si se entra (briefing).

Etiquetas

Etiqueta.astro, del taller de ayer.

Favorito de 44 × 44 px

Un <button> con aria-pressed: hace algo aquí.

La tarjeta recibe la receta: no va a buscar datos.

Un solo enlace

El del título. Su ::after cubre toda la tarjeta.

Toda la tarjeta clicable, con un solo enlace

```css
.tarjeta {
  position: relative;
}

.tarjeta__enlace::after {
  content: "";
  position: absolute;
  inset: 0;
  z-index: 1;
}

.tarjeta__favorito {
  z-index: 2;  /* queda encima */
}
```



Un lector de pantalla oye un enlace por tarjeta, no tres.

Error intencional: la imagen sin width y height

```
<img
  src={`${base}${imagen}`}
  alt=""
  width="400"
  height="300"
  loading="lazy" />
```

CLS (Cumulative Layout Shift): cuánto se mueve la página mientras carga. Lighthouse lo mide y lo penaliza.

1. Borra width y height
2. DevTools → Network → Slow 3G
3. Recarga: el texto salta cuando llega la imagen
4. Vuelve a ponerlos: el espacio queda reservado

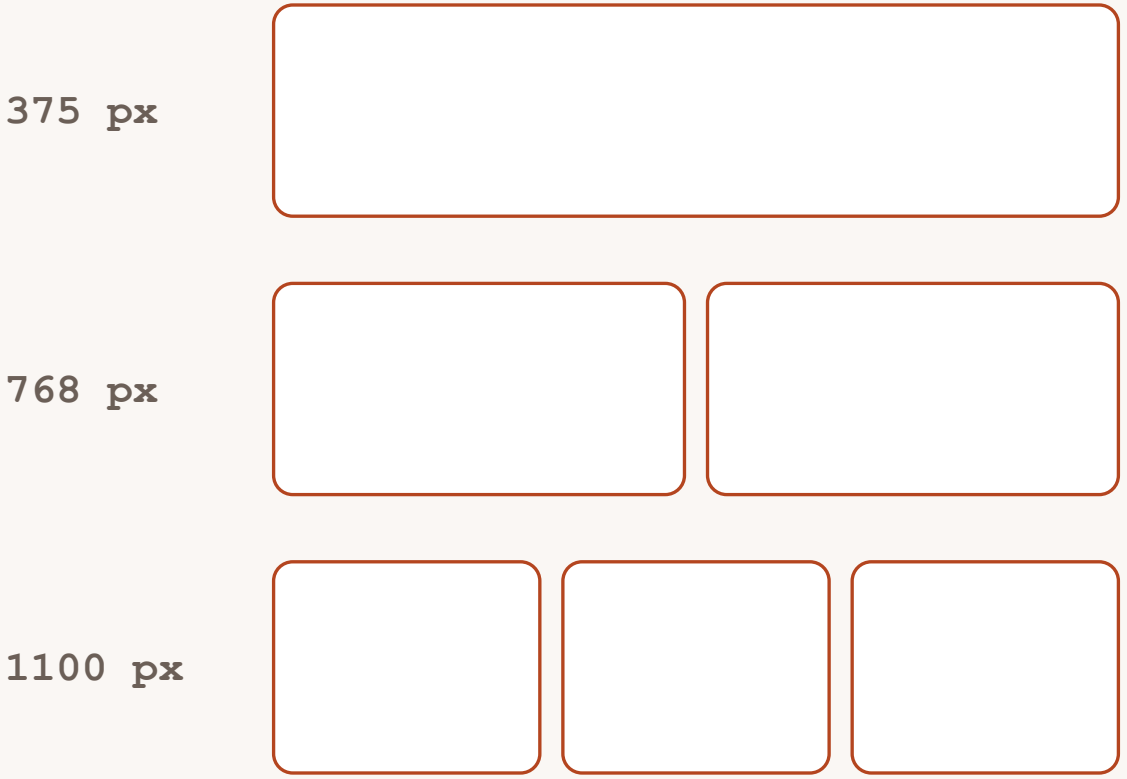
Con width y height el navegador reserva el espacio antes de descargar la imagen.

Una línea de Grid reemplaza tres media queries

```
/* antes */
.rejilla { grid-template-columns: 1fr; }
@media (min-width: 600px) { … 2 … }
@media (min-width: 1000px) { … 3 … }
```

```
/* ahora */
grid-template-columns:
  repeat(auto-fit,
         minmax(260px, 1fr));
```

Caben tantas columnas de 260 px como permita el ancho, y lo que sobra se reparte.



auto-fit estira las columnas para llenar el ancho; auto-fill deja huecos vacíos si hay pocas tarjetas.

Cómo bajan los datos: página → CardGrid → Card

datos-de-prueba.js

`export const recetas`

→

index.astro

`recetas.slice(0, 3)`

→

CardGrid

`recetas.map(...)`

→

Card

`una receta`

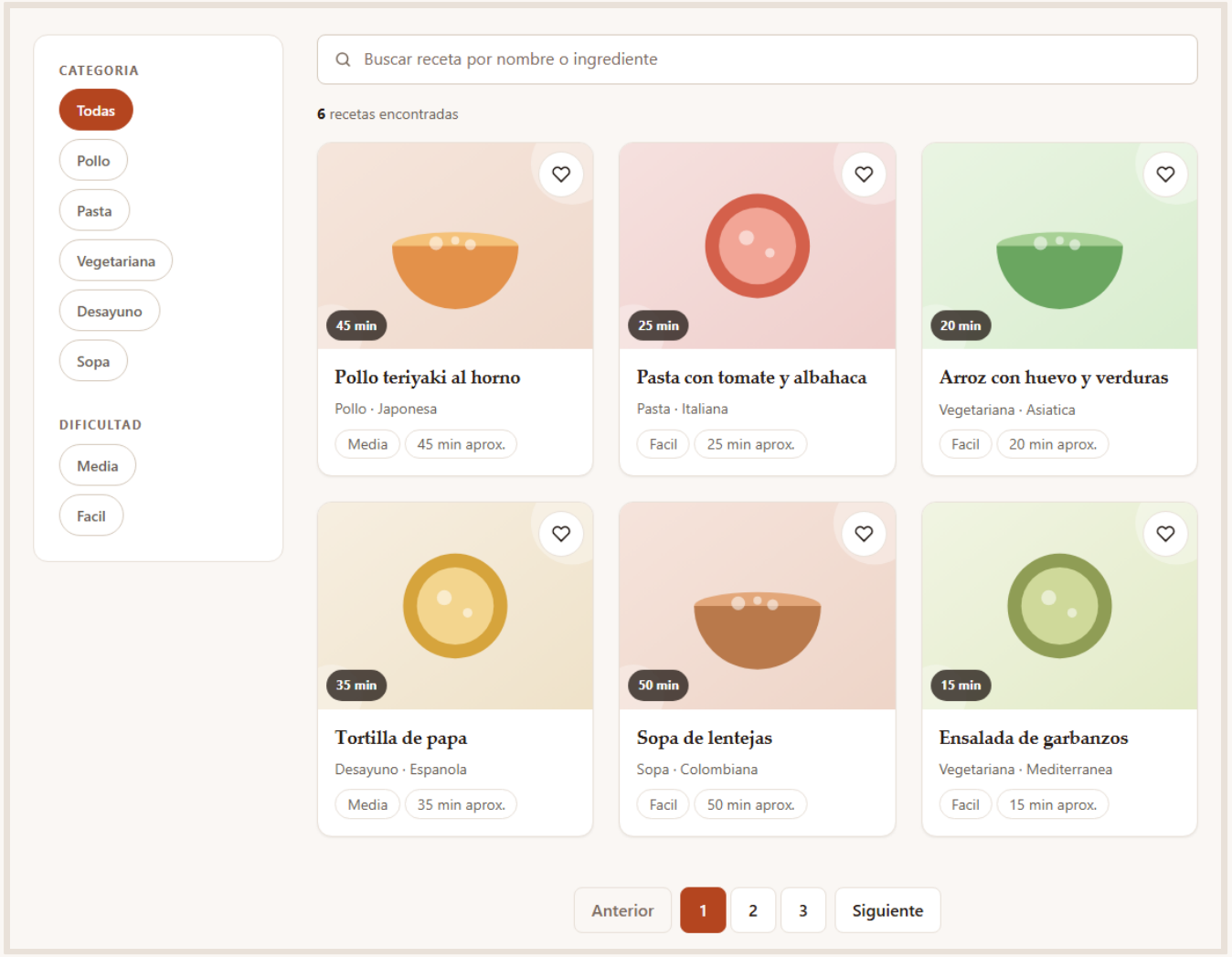



```
<CardGrid
  recetas={
    recetas
    .slice(0, 3)
  } />
```

Cada componente recibe solo lo que necesita.

PASOS 4 Y 5

La página del catálogo





Celular: filtros en fila arriba. Escritorio: barra lateral de 240 px.

PASO 4

Buscador, filtros y paginación: se ven, aún no funcionan

BarraDeBusqueda

- <label> oculto con solo-lectores
- El placeholder no reemplaza al label
- role="search"

Filtros

- Son <button>: cambian esta página
- aria-pressed marca el activo
- El CSS usa [aria-pressed="true"]

Paginacion

- aria-current="page", como en el Header
- disabled en "Anterior"
- Array.from crea [1, 2, 3]

```
.filtro[aria-pressed="true"] {
  background: var(--color-marca);  color: #ffffff;
}
```

PASO 5

Error intencional: min-width: 0

```
.catalogo__resultados {
  min-width: 0;
}

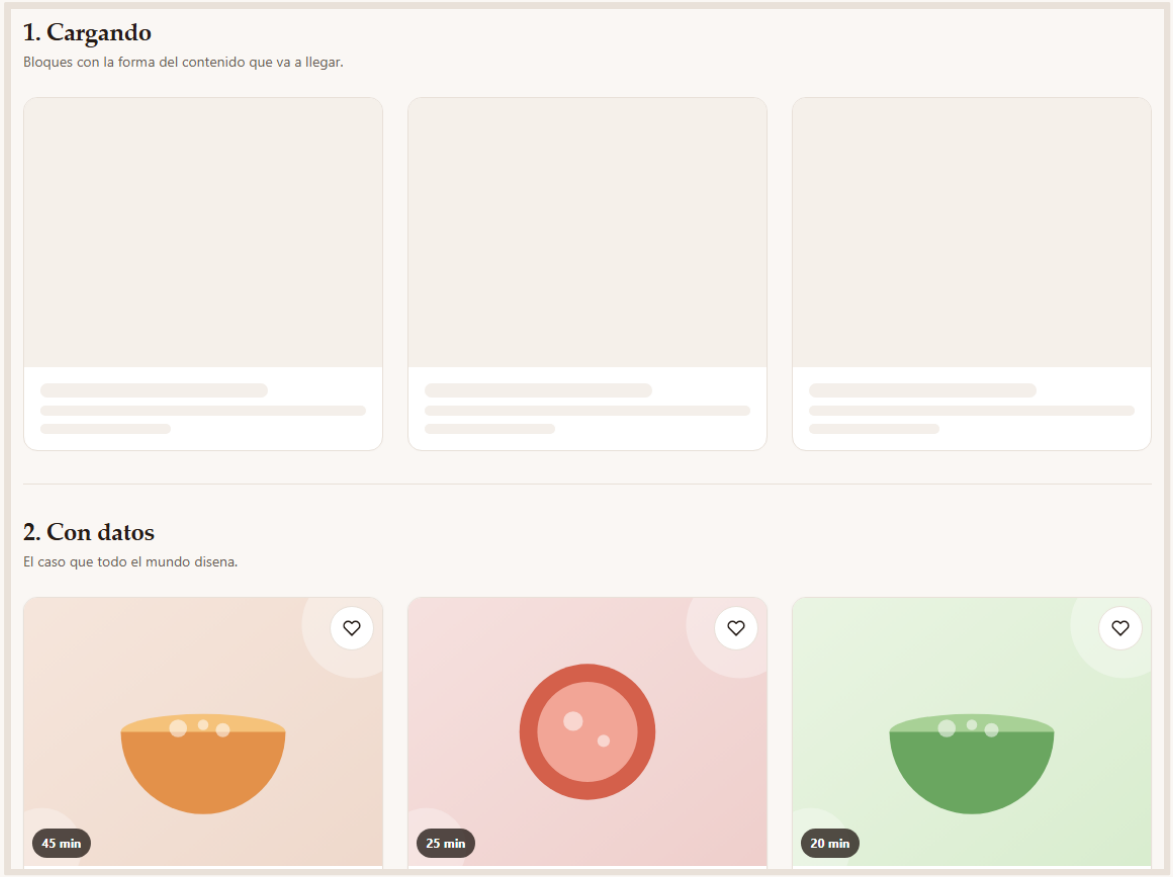
.filtros {
  min-width: 0;
}
```

La fila de filtros es más ancha que la pantalla y empuja su columna.

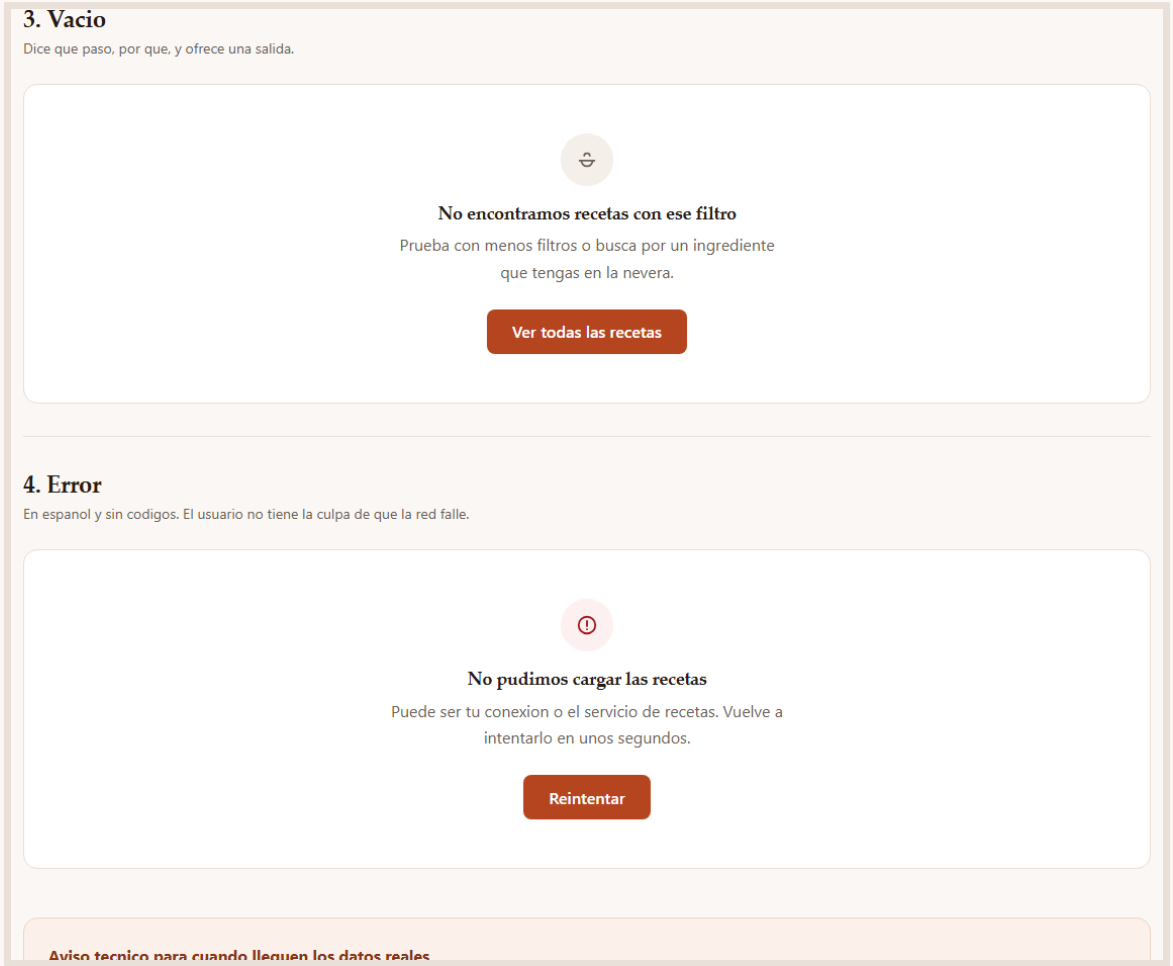
1. Borra los dos min-width: 0
2. Simulador a 320 px
3. Consola: scrollWidth === clientWidth
4. Responde false: la página se mueve de lado
5. Vuelve a ponerlos: true

Un hijo de Grid o Flexbox no se encoge por debajo de su contenido, a menos que le digamos min-width: 0.

Toda pantalla con datos existe en cuatro versiones



1. Cargando · 2. Con datos



3. Vacío · 4. Error

Estado: vacío y error son el mismo componente

```
<Estado
  tipo="vacio"
  titulo="No encontramos recetas con ese filtro"
  mensaje="Prueba con menos filtros..."
>
  <Boton href={`${base}catalogo`}>
    Ver todas las recetas
  </Boton>
</Estado>
```

Aviso: cuando la API no encuentra nada devuelve meals: null. Eso es vacío, no error.

<slot />

La acción entra desde afuera: cada pantalla decide su botón.

role

"status" para el vacío, "alert" para el error: el lector los anuncia.

No solo rojo

El error cambia color, ícono y texto. El color nunca es la única señal.

Esqueleto

Bloques con la forma de la tarjeta, no una rueda girando.

PASO 8

Probar, construir y publicar

1. 320 px: true en Inicio, Catálogo y Estados
2. Teclado: Tab recorre cada tarjeta y su favorito
3. npm run build: 6 páginas, 0 archivos .js
4. push y check verde en Actions
5. Abrir la URL en un celular real

```
npm run build
git add .
git commit -m "feat: catalogo
  con datos de prueba"
git push
```

TALLER EN PAREJAS · ENTREGA ANTES DE LA PRÓXIMA CLASE

Taller: el catálogo de su proyecto

En clase (35 min)

- datos-de-prueba.js: seis elementos de SU dominio, con los campos de SU API
- Card que recibe un elemento por props, sin textos fijos
- Imagen con width, height, alt y proporción fija
- CardGrid con auto-fit y minmax
- Su listado usa CardGrid · build · push

En casa

- Tarjeta clicable con ::after y botón de 44 px encima
- Filtros, buscador y paginación visibles
- Su /estados con los cuatro estados
- Tres destacados en la portada
- 320 px: true

No se copia SaborU: se traduce. Los campos de la tarjeta salen de su API.

PRÓXIMA CLASE

Datos reales desde la API

fetch en el build, una página por receta y un plan B si la API falla. Las tarjetas no cambian: solo cambia de dónde vienen los datos.

Traer: la dirección de su API y una respuesta abierta en el navegador